mySQL

Pros

- Stores data in a lasting relational database that's not easily changed or lost
- Uses disk, so little memory is consumed by accessing or storing it
- More support exists for geographic data types and manipulation

Cons

- Slower to access, may look like lag in loading map results for our users or may cause longer optimization times when adding query latency
- Not as easy to change ranking/weight values for the attractions, which we want users to be able to update as they please and see results quickly

Redis

Pros

- Very quick to access, will allow for less lag in map display of data
- Easy overwrite of ranking/weight values for Boston attractions

Cons

- Uses RAM to store values, so trying to loop through all 10,000+ crime locations for map display or for the gurobi solver will likely fill the computer's memory
- Less support for location/geographic data types
- Data is less secure because it's stored temporarily/only in memory

Recommendations

- Create a mySQL DB to hold all data as a base/backup persistent source of truth
- Use queries to pull the data for a specific map zone when using them for display, then hold them in redis as a cache for that viewpoint, with a set expiration
- Use redis to hold attraction weights given by the user, as these may change frequently. Since there are only 20 values, they will not tax RAM too much.
- Use redis to hold optimization results, since a user may run an optimization once, change attraction values, then switch back to the previous values to compare. There will also not be many of these values, so they will not fill memory.